PROJECT PLAN & DEVELOPMENT TIMELINE

SkillLens

A Skill-to-Employment Intelligence Platform

Project	SkillLens — Final Year Major Project (B.E. Computer Science)
Institute	Swami Vivekanand College of Engineering, Indore
Mentor	Ms. Amisha Jain
Team	Dev1 · Dev2
Duration	15 Development Days	~60 Hours Total Effort
Date	March 2026

1. Project Overview

This document defines the complete development plan for SkillLens — covering work breakdown, task ownership, the day-by-day Gantt schedule, milestones, data asset build plan, and risk management. It is the operational reference for both developers throughout the 15-day build sprint.

Field	Detail
Total duration	15 development days (~60 hours combined effort)
Team size	2 developers — Dev1 and Dev2
Development approach	API-first: backend fully built and Postman-tested before frontend begins
Architecture	MERN stack — MongoDB, Express.js, React.js, Node.js + OpenAI API
Version control	Git with main / dev / feature-branch strategy
Deployment target	Backend → Render free tier | Frontend → Vercel free tier

2. Team Roles and Ownership

Dev1 **Dev2** **Both**

	Dev1	Dev2
Primary role	Backend Lead	Frontend Lead + AI & Data
Core modules owned	Auth, Resume API, Quiz Engine, Company Match API, Dashboard API	Interview Module, Roadmap Module, OpenAI integration, all UI pages
Data assets	MongoDB schemas, Mongoose models, Postman collection	questions.json, companies.json, resources.json, interview_questions.json
Deployment	Render backend deployment, environment variables	Vercel frontend deployment, final integration testing
Cross responsibility	Code reviews on frontend PRs; assist with UI bugs in Week 3	Code reviews on backend PRs; assist with API bugs in Week 2

Either developer must be able to continue the other's work from documentation alone. All module logic is documented in the Module-Based Project Document.

3. Phase-by-Phase Development Plan

The project is divided into 7 sequential phases. Backend phases (1–4) run before frontend phases (5–6) to ensure stable APIs exist before UI work begins. Data asset creation runs in parallel throughout.

Phase 1 — Project Setup (Days 1–2)

Task	Owner	Hrs	Output
Initialise Git repo; define branching strategy (main / dev / feature/*)	Both	1	GitHub repo live
Backend scaffold: Express, Mongoose, dotenv, folder structure	Dev1	2	server.js running, DB connected
Frontend scaffold: React, Tailwind CSS, React Router, Axios setup	Dev2	2	React app running with routing
Set up Postman collection with environment	Dev1	1	postman_collection.json

variables			
MongoDB Atlas cluster creation + .env config	Dev1	1	Live Atlas connection confirmed
Agree and document final folder structure for both repos	Both	1	Folder structure doc committed

Phase 2 — Authentication (Days 2–3)

Task	Owner	Hrs	Output
User Mongoose schema (name, email, password, targetRole, timestamps)	Dev1	1	models/User.js
POST /api/auth/register — validation, bcrypt hash, JWT response	Dev1	2	Register endpoint tested in Postman
POST /api/auth/login — bcrypt compare, JWT issuance	Dev1	2	Login endpoint tested in Postman
JWT auth middleware — token extraction, verification, req.user attach	Dev1	1.5	middleware/authMiddleware.js
GET /api/auth/profile and PUT /api/auth/profile endpoints	Dev1	1	Profile endpoints live
Login.jsx and Register.jsx — form, validation, error display	Dev2	3	Auth pages functional
AuthContext.jsx — token storage, user state, protected route wrapper	Dev2	2	App-wide auth state working

Phase 3 — Resume Analyzer + Skill Quiz Engine (Days 4–7)

Task	Owner	Hrs	Output
Build questions.json — 100 MCQs (10 skills × 10 questions, with answers)	Dev2	7	data/questions.json
Multer upload middleware — memory storage, PDF only, 5 MB limit	Dev1	1	middleware/uploadMiddleware.js
pdf-parse text extraction service	Dev1	1.5	services/resumeService.js
Skill detection logic — 30-skill master list, string matching	Dev1	2	detectSkills() in resumeService
POST /api/resume/upload endpoint	Dev1	1	Resume endpoint tested in Postman
SkillProfile Mongoose schema	Dev1	1	models/SkillProfile.js
GET /api/quiz/:skill — load + shuffle 10 questions, strip answers	Dev1	1.5	Quiz fetch endpoint
POST /api/quiz/:skill/submit — score, classify depth, upsert profile	Dev1	2	Quiz submit endpoint + scoring logic
Trigger recalculations after quiz submit (match % + Readiness Score)	Dev1	1	Integrated into quizController
ResumeUpload.jsx — drag-and-drop, skill results panel	Dev2	3	Resume upload page
Quiz.jsx — question display, option selection, progress indicator	Dev2	4	Quiz page functional
SkillCard.jsx — depth score display with level badge	Dev2	2	Reusable skill card component

Phase 4 — Company Match Engine (Days 7–9)

Task	Owner	Hrs	Output

Build companies.json — 25 companies with skill weights and min depth	Dev2	8	data/companies.json
Weighted match scoring service — depth-aware, per-company	Dev1	2.5	services/scoringService.js
Gap analysis logic — top 3 skills by impact, impact simulation	Dev1	2	calculateGaps() in scoringService
GET /api/companies — all 25 companies with current match %	Dev1	1	Company list endpoint
GET /api/companies/:id — full profile with student depth overlay	Dev1	1	Company detail endpoint
GET /api/companies/:id/gap — gap analysis with verdicts	Dev1	1	Gap analysis endpoint
CompanyMatch.jsx — ranked list with colour-coded readiness bars	Dev2	3	Company match page
CompanyDetail.jsx — gap breakdown, top 3 skills, verdict card	Dev2	3	Company detail page

Phase 5 — AI Mock Interview (Days 9–11)

Task	Owner	Hrs	Output
Build interview_questions.json — 5 questions × 10 roles	Dev2	3	data/interview_questions.json
OpenAI service — structured prompt, JSON parse, error handling	Dev2	4	services/openaiService.js
InterviewResult Mongoose schema	Dev1	1	models/InterviewResult.js
GET /api/interview/questions/:role endpoint	Dev1	0.5	Questions endpoint
POST /api/interview/evaluate — call OpenAI, store result	Dev1	2	Evaluate endpoint
Keyword feedback loop — missedKeywords → SkillProfile weakFlag	Dev1	1.5	Cross-module signal working
GET /api/interview/history endpoint	Dev1	0.5	History endpoint
Interview.jsx — question display, answer textarea, submit flow	Dev2	3	Interview page
FeedbackCard.jsx — 6 separate feedback fields, keyword tags	Dev2	3	Feedback display component
Graceful degradation UI — loading state, timeout message	Dev2	1	Error state handled in UI

Phase 6 — Roadmap + Dashboard (Days 11–13)

Task	Owner	Hrs	Output
Build resources.json — 10 skills × 2 levels × 4 resources = 80 entries	Dev2	6	data/resources.json
Roadmap generation service — AI ordering + resource fill from JSON	Dev2	4	services/roadmapService.js
RoadmapProgress Mongoose schema	Dev1	1	models/RoadmapProgress.js
POST /api/roadmap/generate endpoint	Dev1	1	Generate endpoint
GET /api/roadmap endpoint	Dev1	0.5	Fetch roadmap endpoint
PUT /api/roadmap/:week/complete — mark done, trigger recalc	Dev1	1	Completion endpoint
Readiness Score service — 4-component weighted formula, history save	Dev1	2.5	services/readinessService.js
GET /api/dashboard/metrics — single aggregated response	Dev1	1.5	Dashboard endpoint
Roadmap.jsx — weekly cards, resource links, task checkbox	Dev2	4	Roadmap page
Dashboard.jsx — Readiness Score hero, charts, company list	Dev2	4	Dashboard page
ChartComponent.jsx — line chart (score history) + bar chart (skills)	Dev2	2	Recharts components

Phase 7 — Integration, Testing, Deployment (Days 13–15)

Task	Owner	Hrs	Output
End-to-end integration test — full user flow from register to roadmap	Both	4	Bug list
Fix integration bugs	Both	3	Stable build
Postman API test — all endpoints verified against expected responses	Dev1	3	Completed API test checklist

UI cross-browser test — Chrome, Firefox, Edge, mobile viewport	Dev2	3	UI test results
Readiness Score recalculation verification across all trigger points	Dev1	2	Score updates confirmed
Deploy backend to Render — env vars, CORS config	Dev1	2	Live backend URL
Deploy frontend to Vercel — env vars, production API URL	Dev2	1.5	Live frontend URL
Full demo walkthrough on deployed URLs	Both	2	Demo-ready application
Final documentation review and submission package	Both	1.5	All docs finalised

4. Gantt Schedule — 15-Day Sprint

Colour key: **Dev1** **Dev2** **Both**

Task	Owner	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Git repo + branching setup	Both															
Backend scaffold	Dev1															
Frontend scaffold	Dev2															
Postman + Atlas setup	Dev1															
User schema + Auth endpoints	Dev1															
JWT middleware	Dev1															
Login & Register UI + AuthContext	Dev2															
questions.json (100 MCQs)	Dev2															
Resume upload + pdf-parse service	Dev1															
Skill detection logic	Dev1															
SkillProfile schema + Quiz endpoints	Dev1															
Quiz recalculation integration	Dev1															
Resume Upload UI	Dev2															
Quiz UI + SkillCard component	Dev2															
companies.json (25 profiles)	Dev2															
Scoring + gap analysis service	Dev1															
Company Match API endpoints	Dev1															
Company Match + Detail UI	Dev2															
interview_questions.json	Dev2															
OpenAI service + structured prompt	Dev2															
InterviewResult schema + endpoints	Dev1															
Keyword feedback loop	Dev1															
Interview UI + FeedbackCard	Dev2															
resources.json (80 entries)	Dev2															
Roadmap service + AI ordering	Dev2															
Roadmap schemas + endpoints	Dev1															

Readiness Score service	**Dev1**															
Dashboard API endpoint	**Dev1**															
Roadmap UI	**Dev2**															
Dashboard + Charts UI	**Dev2**															
End-to-end integration test	**Both**															
Bug fixes	**Both**															
API Postman tests	**Dev1**															
UI cross-browser tests	**Dev2**															
Backend deployment (Render)	**Dev1**															
Frontend deployment (Vercel)	**Dev2**															
Final demo + docs review	**Both**															

Shaded cells indicate active development days. White cells indicate the task has not yet started or is complete. Day numbers are relative to the sprint start date.

5. Project Milestones

#	Milestone	Target Day	Owner	Completion Criteria
M1	Project Setup Complete	Day 2	Both	Frontend and backend running locally; Atlas connected; Git repo live with branch strategy
M2	Authentication Working	Day 3	Dev1 + Dev2	Register, login, JWT verified in Postman; Login/Register UI functional with error handling
M3	Resume + Quiz Complete	Day 7	Dev1 + Dev2	PDF upload extracts skills; quiz scores stored; depth level displayed in UI; recalculation triggers working
M4	Company Match Working	Day 9	Dev1 + Dev2	25 companies in JSON; weighted match % calculated; gap analysis with top-3 skills displayed in UI
M5	Interview Module Complete	Day 11	Dev1 + Dev2	OpenAI returns structured JSON; all 6 feedback fields displayed separately; keyword loop updating SkillProfile
M6	Roadmap + Dashboard Complete	Day 13	Dev1 + Dev2	Roadmap generates with real resource links; Readiness Score updates across all trigger points; dashboard charts render
M7	Deployed and Demo-Ready	Day 15	Both	App live on Vercel + Render; all flows work end-to-end on deployed URLs; no critical bugs

6. Data Asset Build Plan

Three static JSON files are the core intelligence of SkillLens. They are manually curated by Dev2, running in parallel with backend development. These files must be ready before the modules that depend on them begin API integration.

Asset	Owner	Build Window	Size	Source / Method
questions.json (Quiz question bank)	Dev2	Days 4–6 (parallel with Phase 3)	100 questions (10 skills × 10 MCQs)	Written by team; each question validated against GFG, official docs, and standard CS curriculum. Each MCQ has 4 options and one marked correct answer.
companies.json (Company profiles)	Dev2	Days 7–9 (parallel with Phase 4)	25 company profiles with skill weights	Researched from 8–10 real job postings per company on Naukri and LinkedIn. Required skills and weights assigned based on frequency of appearance across postings.
resources.json (Resource library)	Dev2	Days 11–12 (parallel with Phase 6)	80 entries (10 skills × 2 levels × 4 resources)	One YouTube video, one article (GFG / official docs), one practice link (LeetCode / HackerRank / platform), and one concrete build task per skill per level. Curated for quality over quantity — one best resource per slot.
interview_questions.json (Interview question bank)	Dev2	Days 9–10 (parallel with Phase 5)	50 questions (10 roles × 5 questions)	Role-specific questions covering core technical concepts. Each question tagged with a topic (e.g. auth, middleware, DSA) to support keyword feedback mapping.

Data assets are version-controlled in the Git repo under /backend/data/. They can be updated without code changes or redeployment.

7. Risk Register

Risk	Prob.	Impact	Owner	Mitigation
OpenAI API costs exceed budget during testing	Low	Medium	Dev2	Use mock AI responses during development; only call live API for final integration testing. Set a hard usage cap in the OpenAI dashboard.
One developer unavailable for 2+ consecutive days	Medium	High	Both	All module logic documented in Module-Based Document. Either developer can continue the other's work. Cross-review all PRs so both devs understand every module.
companies.json takes longer than 8 hours to build	Medium	Medium	Dev2	Start with 10 companies for MVP; expand to 25 progressively. Naukri research can be done in 1-hour blocks between backend tasks.
pdf-parse fails on certain resume formats	Medium	Low	Dev1	Prepare 3 verified test resumes (text-based PDF). Display a clear error message in UI: "Please upload a text-based PDF."
MongoDB aggregation more complex than estimated	Medium	Low	Dev1	Readiness Score calculation can be done in Node.js service logic instead of DB aggregation if Mongoose aggregation proves too complex within timeline.
Render free tier cold start delays break demo	High	Low	Dev1	Keep a browser tab open to the backend health endpoint before demo. Use a ping

				service (UptimeRobot free) to keep the dyno warm.
OpenAI API returns non-JSON or truncated response	Medium	Medium	Dev2	Wrap JSON.parse() in try-catch; return 503 with stored answer if parsing fails. Test with adversarial answers (very long, all symbols) before demo.

8. Definition of Done

A task is considered complete only when ALL of the following are true:

- The feature works as specified in the SRS and Feature Document
- The API endpoint returns the correct response, verified in Postman
- The UI displays the data correctly and handles empty and error states
- The code is committed to the dev branch with a meaningful commit message
- The other developer has reviewed and approved the pull request
- No console errors or unhandled promise rejections in the browser or terminal

A phase is complete only when all tasks in that phase meet the Definition of Done AND the phase milestone criteria are confirmed by both developers.